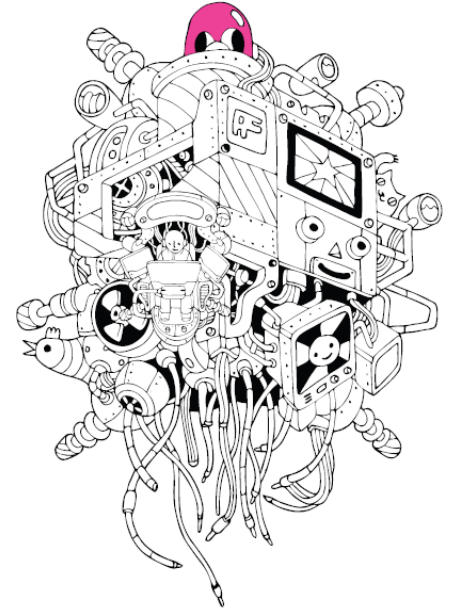


C programming



C Programming:

Chapter 04. Understanding Data Representation Methods

C programming

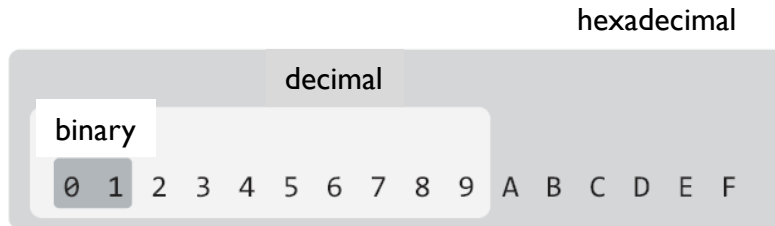


Chapter 04-1. How computers represent data

C Programming:

What is binary?

What are decimal and hexadecimal ?



Binary

- A method of expressing values(data) using two symbols

Decimal

- A method of expressing values(data) using ten symbols

N base

- A method of expressing values(data) using N symbols

	decimal	hexadecimal
Increase the number of digits	9	9
	10	A
	11	B
	12	C
	13	D
	14	E
	15	F
	16	10
17	11	

																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											</
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

decimal	binary
0	0
1	1
2	10
3	11
4	100
5	101



Bit and Byte, which are units of data representation

1 bit



1 byte

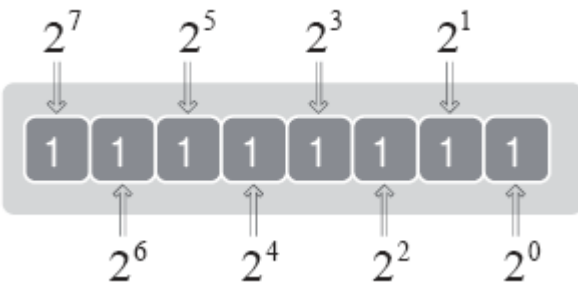


2 byte



The address value of computer memory is as signed **one address per byte**.

Therefore, a byte is a very meaningful unit for a computer.



Using the meaningful information on the left, you can easily convert binary to decimal.

Data representation using octal and hexadecimal

```
int num1 = 10;    // If there is no special declaration, it is a decimal representation
int num2 = 0xA;   // If it starts with 0x, it is recognized as hexadecimal
int num3 = 012;   // If it starts with 0, it is recognized as an octal number
```

```
int main(void)
{
    int num1=0xA7, num2=0x43;
    int num3=032, num4=024;

    printf("Decimal integer value of 0xA7: %d \n", num1);
    printf("Decimal integer value of 0x43: %d \n", num2);
    printf("Decimal integer value of 032: %d \n", num3);
    printf("024 decimal integer value: %d \n", num4);

    printf("%d-%d=%d \n", num1, num2, num1-num2);
    printf("%d+%d=%d \n", num3, num4, num3+num4);
    return 0;
}
```

Execution results

```
Decimal integer value of 0xA7: 167
Decimal integer value of 0x43: 67
    Decimal integer value of 032: 26
    Decimal integer value of 024: 20
167-67=100
26+20=46
```



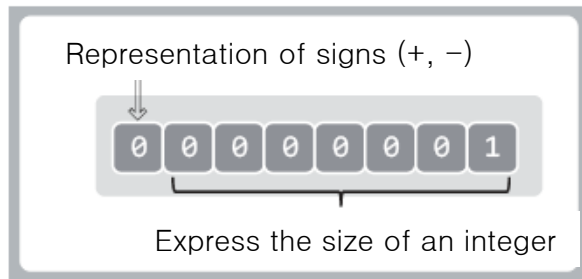
C programming



Chapter 04-2. Representation of integers and real numbers

C Programming:

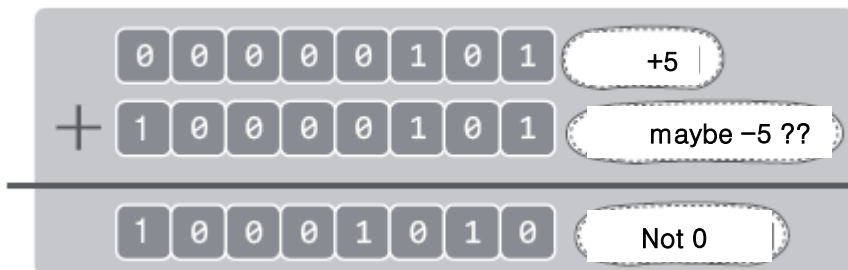
Positive integer representation



- The leftmost bit is called **MSB (Most Significant Bit)**.
- **MSB** is the bit that indicates the sign.
- The remaining bits except for the **MSB** are used to indicate the size.

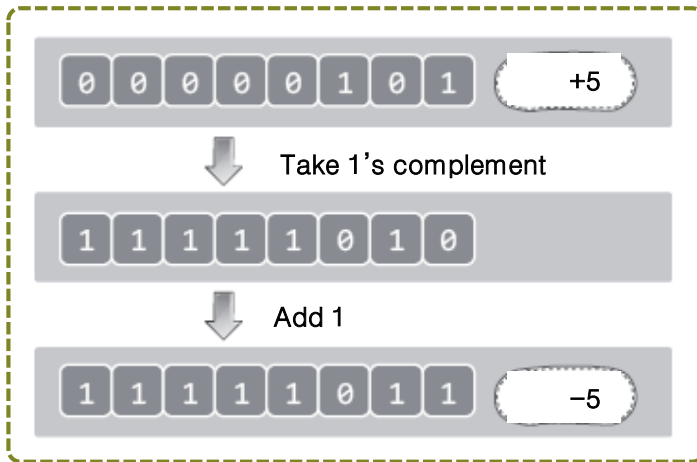
The representation of integers is independent of their byte size.

That is; The larger the byte size, the wider the range of integers it can represent.



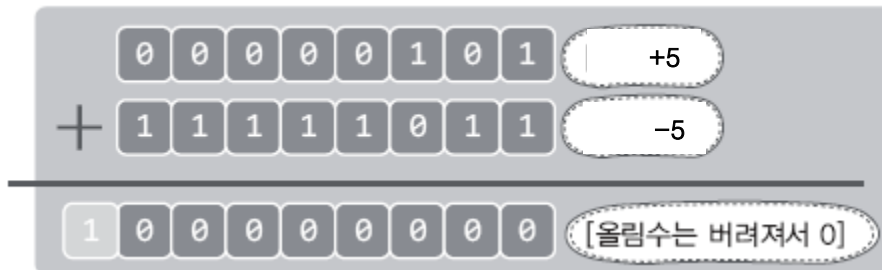
we explain why it is not appropriate to represent negative integers in the same way as positive integers.

Negative integer representation



A way to represent negative integers

Negative integers are represented by taking the 2's complement.



2's complement expression

Check if it is valid to express negative numbers

The number of rounds is discarded and becomes 0

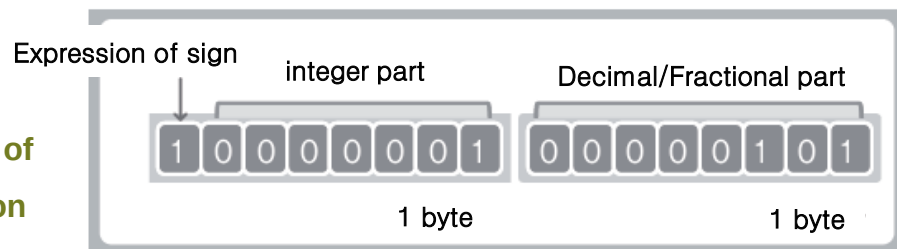


Real number representation

If we represent real numbers in a similar way to how we represent integers, the following problem arises :

- The number of real numbers that can be expressed is limited .
- It cannot even properly represent the infinite number of real numbers between 3.1245 **6** and 3.1245 **7**

Inappropriate method of
real number expression

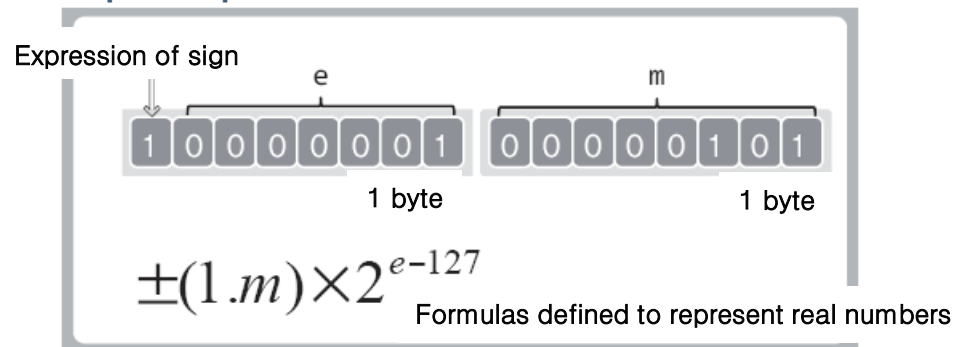


Therefore, the way real numbers are represented is different from the way integers are represented.

- In the representation of real numbers, the range of values that can be expressed is expanded instead of sacrificing precision.
- Therefore, computers cannot express real numbers with perfect precision.

There are errors, but

How to express an appropriate real numbers



Checking for errors in real number expressions

```
int main(void)
{
    int i;
    float num=0.0;

    for(i=0; i<100; i++)
        num+=0.1;    // This operation will be performed a total of 100 times.

    printf("Result of adding 0.1 100 times: %f \n", num);
    return 0;
}
```

Execution results

Adding 0.1 100 times gives: 10.000002

In theory, there is no computing environment capable of perfectly representing all real numbers without error. In other words, the occurrence of errors in real number representation is a fundamental characteristic of computers, not a characteristic of the C language.



C programming



Chapter 04-3. Bitwise Operators

C Programming:

Bitwise operators (bitwise shift operators)

Operator	Function of the Operator	Associativity Direction
&	Performs a bitwise AND operation. Example: num1 & num2;	→
	Performs a bitwise OR operation. Example: num1 num2;	→
^	Performs a bitwise XOR operation. Example: num1 ^ num2;	→
~	A unary operator that inverts (flips) all bits of the operand. Example: ~num; // num itself does not change; only the inverted result is returned	←
<<	Shifts the operand's bit pattern to the left . Example: num << 2; // num does not change; returns the result shifted left by two positions	→
>>	Shifts the operand's bit pattern to the right . Example: num >> 2; // num does not change; returns the result shifted right by two positions	→



& operator : bitwise AND

```
int main(void)
{
    int num1 = 15;    // 00000000 00000000 00000000 00001111
    int num2 = 20;    // 00000000 00000000 00000000 00010100
    int num3 = num1 & num2;    // Bitwise & operation of num1 and num2
    printf("Result of AND operation: %d \n", num3);
    return 0;
}
```

AND Result of operation: 4

Execution results

0 & 0	return 0
0 & 1	return 0
1 & 0	return 0
1 & 1	return 1

	00000000	00000000	00000000	000	01111
& 연산	00000000	00000000	00000000	000	10100
	00000000	00000000	00000000	000	00100

| Operator : bitwise OR

```
int main(void)
{
    int num1 = 15;    // 00000000 00000000 00000000 00001111
    int num2 = 20;    // 00000000 00000000 00000000 00010100
    int num3 = num1 | num2;
    printf("Result of OR operation: %d \n", num3);
    return 0;
}
```

OR Result of operation: 31

Execution results

0		0	return	0
0		1	return	1
1		0	return	1
1		1	return	1

	00000000	00000000	00000000	000	01111
연산	00000000	00000000	00000000	000	10100
	00000000	00000000	00000000	000	11111

^ Operator : Bitwise XOR

```
int main(void)
{
    int num1 = 15;    // 00000000 00000000 00000000 00001111
    int num2 = 20;    // 00000000 00000000 00000000 00010100
    int num3 = num1 ^ num2;
    printf("Result of XOR operation: %d \n", num3);
    return 0;
}
```

Result of XOR operation: 27

0 ^ 0	return 0
0 ^ 1	return 1
1 ^ 0	return 1
1 ^ 1	return 0

Execution results

	00000000	00000000	00000000	000	01111
^ 연산	00000000	00000000	00000000	000	10100
	00000000	00000000	00000000	000	11011

~ operator

```
int main(void)
{
    int num1 = 15;    // 00000000 00000000 00000000 00001111
    int num2 = ~num1;
    printf("Result of NOT operation: %d \n", num2);
    return 0;
}
```

~0	return 1
~1	return 0

Result of NOT operation: -16

Execution results

~ Before operation 00000000 00000000 00000000 00001111

~ After operation 11111111 11111111 11111111 11110000



<< Operator : Bit shift left (Shift)

- `num1 << num2` Returns the result of shifting the bit string of num1 to the left by num2 spaces
- `8 << 2` Returns the result of shifting the bit string of integer 8 to the left by 2 spaces

```
int main(void)
{
    int num = 15;    // 00000000 00000000 00000000 00001111

    int result1 = num<<1;    // Shift the bit string of num by 1 space to the left
    int result2 = num<<2;    // Shift the bit string of num by 2 spaces to the left
    int result3 = num<<3;    // Shift the bit string of num 3 spaces to the left

    printf("1 space movement result: %d \n", result1);
    printf("2-space movement result: %d \n", result2);
    printf("3-space movement result: %d \n", result3);
    return 0;
}
```

Result of moving 1 space: 30

Result of moving 2 spaces: 60

Result of moving 3 spaces: 120

Execution results

```
00000000 00000000 00000000 00011110 // 30 in decimal
00000000 00000000 00000000 00111100 // 60 in decimal
00000000 00000000 00000000 01111000 // 120 in decimal
```

Each time you move one space to the left, the value of the integer **doubles**.

Conversely, each time you move one space to the right, the value of the integer **is halved**.



>> Operator : Bit shift right (Shift)

```
11111111 11111111 11111111 11110000    // -16
```



The results of operations vary depending on

```
00111111 11111111 11111111 11111100    // 0이 채워진 경우
```

```
11111111 11111111 11111111 11111100    // 1이 채워진 경우
```

the left bits are filled with 0, the result of shifting the bits to the right varies depending on the CPU.

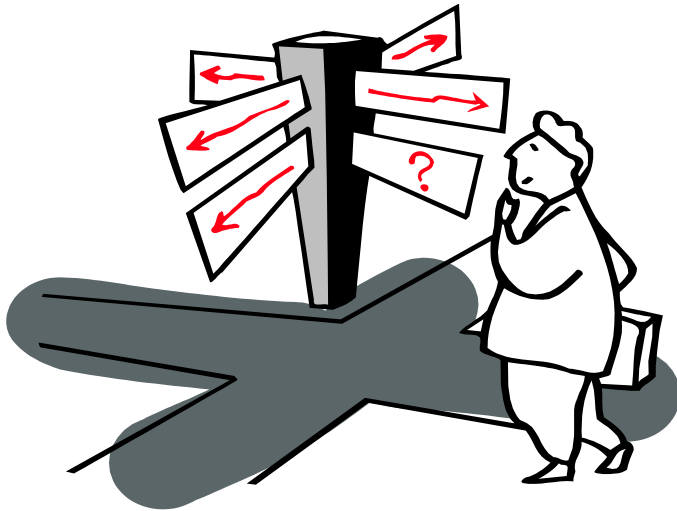
>> operator should be restricted when compatibility is required .

```
int main(void)
{
    int num = -16;    // 11111111 11111111 11111111 11110000
    printf("Result of moving 2 spaces to the right: %d \n", num>>2);
    printf("Result of moving 3 spaces to the right: %d \n", num>>3);
    return 0;
}
```

Execution results

Result of moving 2 spaces right: -4

Result of moving 3 spaces right: -2



Chapter 04 That's it . Do you have
any questions ?